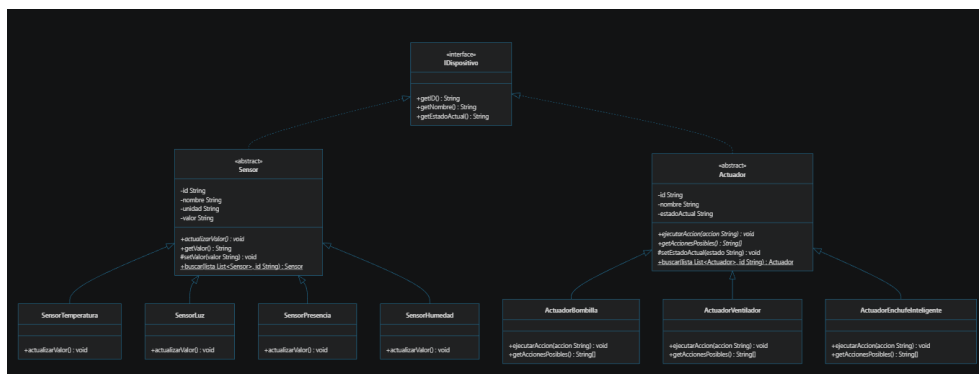


Decisiones de Diseño

En cuanto a la jerarquía diseñada para los dispositivos, la interfaz `IDispositivo` define el contrato mínimo común para todo dispositivo. Incluye los métodos `getID()`, `getNombre()` y `getEstadoActual()`. Las clases abstractas `Sensor` y `Actuador` añaden la lógica compartida entre Sensores (unidad de medida, último valor) y Actuadores (estado actual, validación de acciones) dejando la lógica específica para las implementaciones de las subclases concretas.

Además, Cada clase abstracta incluye un método estático `buscar(List, String)` que localiza un dispositivo por ID dentro de su propia lista. Esta decisión coloca la operación donde semánticamente pertenece: `Sensor` sabe buscar sensores y `Actuador` sabe buscar actuadores.



En cuanto a las reglas, se establece una interfaz `IRegla`, de la cual todas las reglas que se implementen heredarán.



En cuanto a los principios **SOLID** empleados

Principio	Aplicación
S — Responsabilidad única	SmartTecnHouse gestiona colecciones; ServicioLog gestiona el log; GestorPersistencia gestiona la persistencia. Cada clase tiene una sola responsabilidad clara.
O — Abierto/Cerrado	Añadir un nuevo sensor o actuador solo requiere crear una subclase de Sensor o Actuador y registrarla en SmartTecnHouse.inicializar(). No se modifica ninguna clase existente.
L — Sustitución de Liskov	Cualquier subclase de Sensor puede usarse en cualquier lugar donde se espere un Sensor. Las reglas operan sobre List<Sensor> sin conocer el tipo concreto.

I — Segregación de interfaces	IDispositivo expone solo lo común a todos los dispositivos. Las reglas no necesitan conocer métodos específicos de sensores o actuadores para sus operaciones de búsqueda.
D — Inversión de dependencias	Las reglas dependen de la interfaz IRegla, no de implementaciones concretas. SmartTecnoHouse itera List<IRegla> sin conocer qué reglas están activas.

Patrones de Diseño

Patrón Strategy: Se implementa en el Sistema de Reglas. El motivo es que las reglas de negocio deben ser intercambiables. Con este patrón, para añadir una nueva regla basta con crear una nueva clase que la implemente y añadirla a la lista.

Patrón MVC: Implementado en la arquitectura de la aplicación para la segregación de responsabilidades. Este patrón nos permite separar la lógica de negocio de la interfaz gráfica segregando las responsabilidades en tres capas. Modelo con los datos y lógica, sin conocer nada de la vista. Vista, que construye los componentes y no conoce nada del modelo y el Controlado, que es el punto de conexión entre el Modelo y la Vista. Recibe los eventos de la presentación y ejecuta los métodos del modelo.

Patrón Singleton: Aplicado a SmartTecnoHouse y ServicioLog. Ambas clases se instancias de manera única, garantizando que durante todo el ciclo de vida de la aplicación sólo exista una instancia.

Patrón Factory: Para implementación de las reglas existe una aproximación al Patrón Factoría en la que es servicio de persistencia crea las reglas en función del último estado almacenado en el json de persistencia. Es una implementación ligera y basada en los nombres, que debería ser revisada, pero con el fin educativo de la práctica lo considero destacable.

Manual de Usuario

Arranque: Al iniciar la aplicación se carga automáticamente el fichero estado.json (si existe), restaurando el estado de los actuadores y las reglas activas de la sesión anterior.

Lecturas y Ciclos de ejecución: Para efectuar lecturas se debe pulsar el botón 'Simular Lectura'. Eso actualiza los estados de los sensores. El Botón 'Procesar Ciclo Automático' de la parte inferior realiza una simulación de lectura y valida las reglas en consecuencia, cambiando el estado de los actuadores. Este estado también puede modificarse cambiando el estado de los actuadores y clickando en el botón Aplicar. Para activar y desactivar reglas debe usarse los *toggle* (check) de la

parte inferior. Las acciones quedan registradas en el Log de Acciones en la parte derecha.

SmartTecnHouse - Panel de Control

Sensores

Sensor	Estado
Sensor de Temperatura	29.37793739...
Sensor de Luz	105.7623455...
Sensor de Presencia	ON
Sensor de Humedad	26.28347807...

Actuadores

Bombilla ON OFF

Ventilador HIGH HIGH

Enchufe Inteligente OFF OFF

Reglas de Automatización

☒ R1. Ventilación Confortable ☒ R2. Iluminación Automática

Log de Acciones

Timestamp	Actuator	Action	Source
2026-05-28T10:51:25Z	fan	LOW	RULE
2026-05-28T10:51:26Z	bulb	OFF	RULE
2026-05-28T10:51:26Z	fan	HIGH	RULE
2026-05-28T10:51:27Z	bulb	ON	RULE
2026-05-28T10:51:27Z	fan	LOW	RULE
2026-05-28T10:51:28Z	fan	OFF	RULE
2026-05-28T10:51:28Z	bulb	OFF	RULE
2026-05-28T10:51:28Z	fan	HIGH	RULE
2026-05-28T10:51:29Z	fan	OFF	RULE
2026-05-28T10:51:30Z	bulb	ON	RULE
2026-05-28T10:51:36Z	bulb	OFF	MANUAL
2026-05-28T10:51:39Z	fan	HIGH	MANUAL
2026-05-28T10:51:41Z	plug	OFF	MANUAL
2026-05-28T10:51:46Z	bulb	ON	RULE
2026-05-28T10:51:47Z	bulb	OFF	RULE
2026-05-28T10:51:47Z	fan	OFF	RULE
2026-05-28T10:51:48Z	fan	LOW	RULE
2026-05-28T10:51:48Z	fan	HIGH	RULE
2026-05-28T10:51:49Z	bulb	ON	RULE

Cierre: Al cerrar la ventana el estado queda guardado en el archivo estado.json (reglas y estado de los actuadores) y el log sobrescribe el archivo actuators.log

Ampliaciones

Con respecto al punto de partida se añaden dos ampliaciones. **Sensor Humedad**, que mide la humedad relativa con un rango simulado entre 30 y 90% y el **Actuador Enchufe Inteligente**, que permite los estados posibles ON y OFF. Este se apaga automáticamente cuando el sensor de presencia Pir detecta que no hay presencia. Ambas ampliaciones se realizan sin modificar los componentes base, con lo que se demuestra que el modelo es abierto a la extensión, lo que supone un buen diseño.

Conclusiones/Consideraciones

Para la realización de la práctica he tenido que tomar ciertas suposiciones, especialmente al no haber realizado la asignatura de Fundamentos por tenerla convalidada. Plantee la posibilidad de implementar un mecanismo que leyera las mediciones, añadirlo a una cola y simular las distintas medidas, así como leer las medidas anteriores (almacenadas en otro archivo de persistencia), pero considero que con la implementación realizada se demuestran los conocimientos de diseño orientado a objetos. El no haber realizado esa primera práctica me ha supuesto cierta incertidumbre y tener que ir avanzando y deshaciendo hasta que he encontrado un *scope* claro de lo que quería implementar.

El diseño realizado es robusto en cuanto a añadir nuevos elementos (Dispositivos o Reglas) y se han puesto en práctica los conocimientos explicados en las clases. En cuanto mi opinión, he disfrutado mucho realizándola, ya que hacía muchos años que no desarrollaba en Java y esta asignatura me ha permitido reencontrarme con el lenguaje, por lo que estoy muy satisfecho.